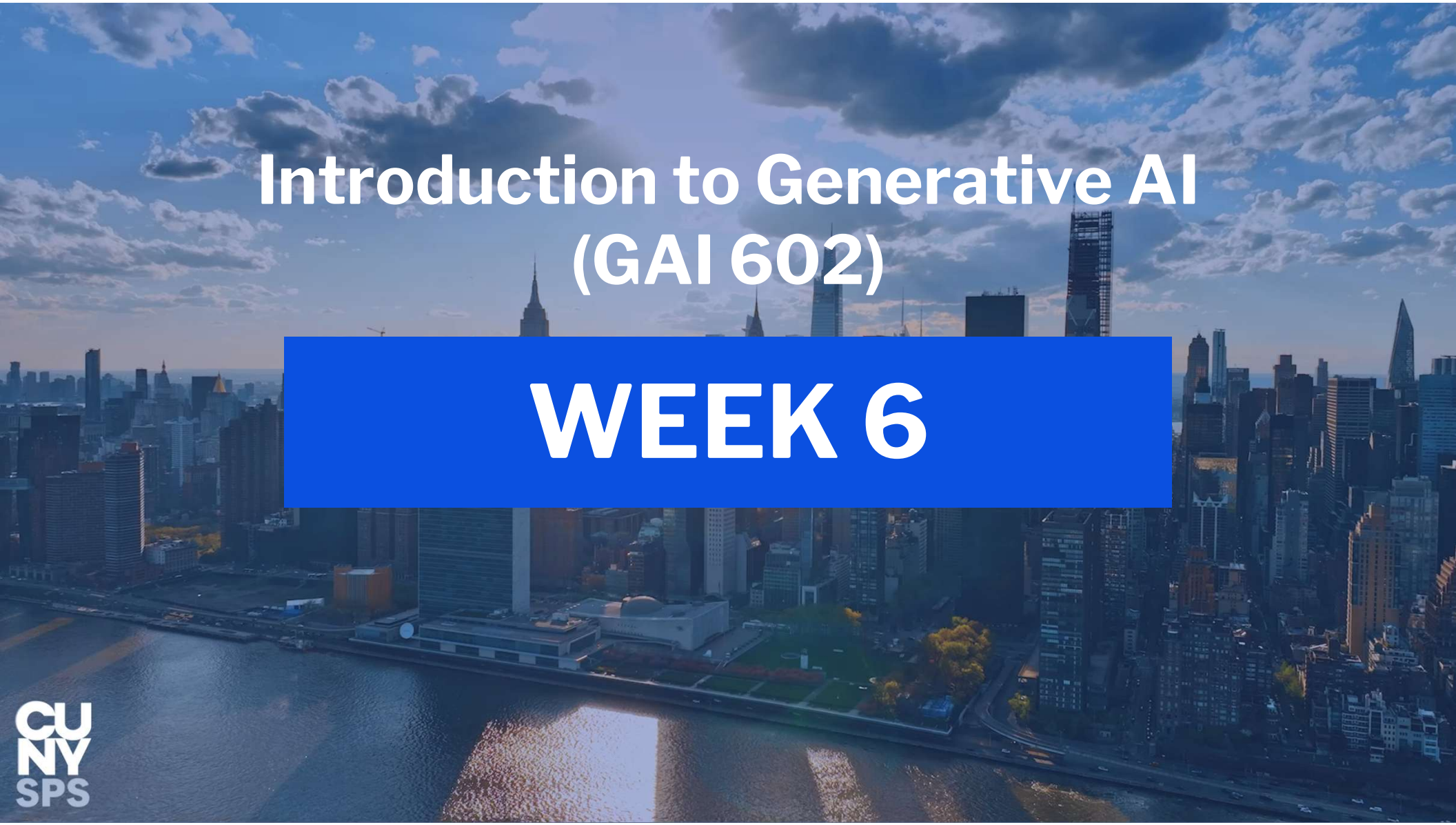

Introduction to Generative AI
(GAI 602)
WEEK 6
CU
NY
SPS


Multi-headed Attention

Self-Attention – Multiheaded Attention

In the paper, the authors also implemented “multi-headed attention” to address the limitations of single-headed attention, which could only focus on one type of relationship or dependency at a time.

Parallel Attention Mechanisms

Multi-headed attention allows the model to focus on different parts of the input sequence simultaneously, capturing diverse relationships (e.g., word-to-word, phrase-to-phrase).

Improved Representations:

Each "head" learns unique attention patterns, enabling the model to extract more nuanced information, such as syntactic and semantic dependencies.

Scalability and Flexibility:

By splitting the computation into smaller attention heads, the model efficiently handles large inputs and produces richer, context-aware embeddings.

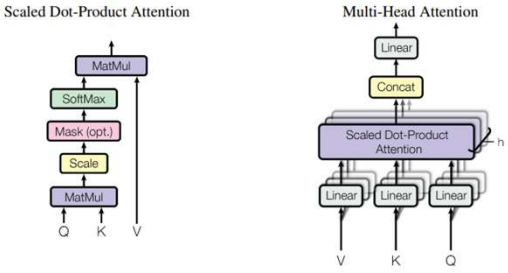
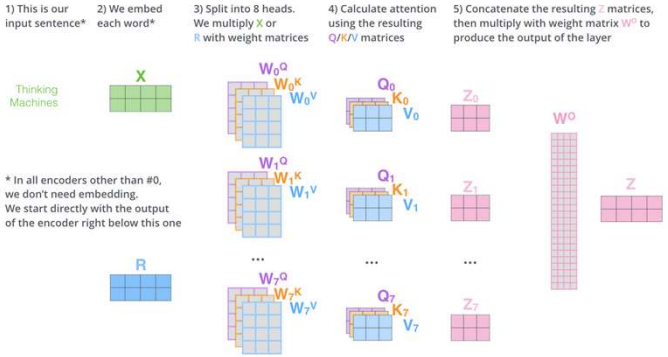


Figure 2: (left) Scaled Dot-Product Attention. (right) Multi-Head Attention consists of several attention layers running in parallel.



Source: Nvidia

Variations of Multiheaded Attention

GQA (Grouped-Query Attention)
Reduces the computational cost by grouping multiple queries together and computing attention collectively rather than individually
Key Advantage: Drastically lowers memory and compute requirements, especially for large-scale models like GPT-3

Multi-Query Attention:
Instead of having separate key-value pairs for each query, all queries share the same set of key-value pairs
Key Advantage: Reduces the required memory for storing multiple key-value pairs without impacting model performance

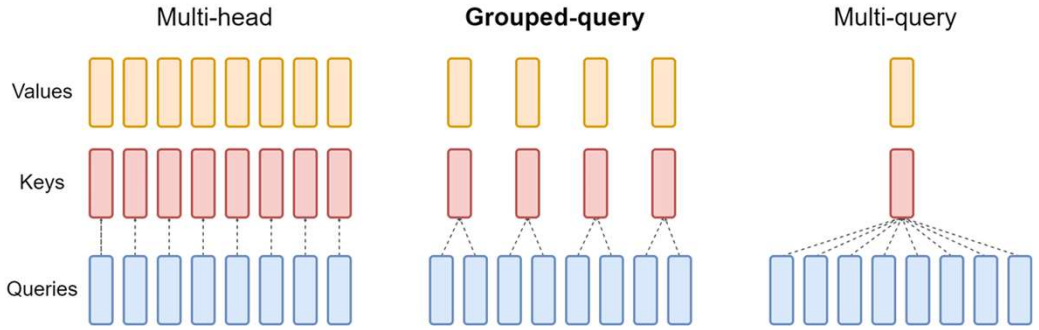


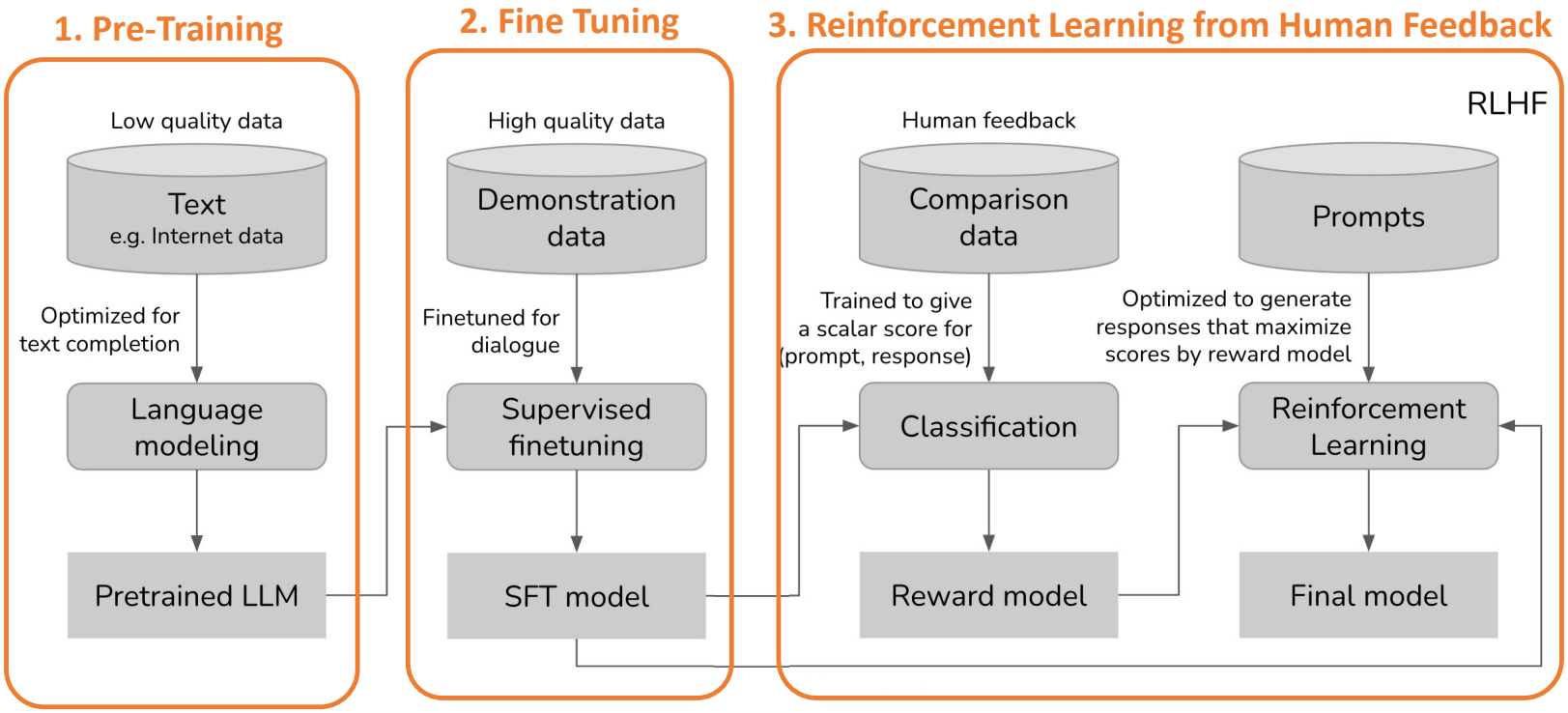
Figure 2: Overview of grouped-query method. Multi-head attention has H query, key, and value heads. Multi-query attention shares single key and value heads across all query heads. Grouped-query attention instead shares single key and value heads for each *group* of query heads, interpolating between multi-head and multi-query attention.

Source: Nvidia



Pre-Training LLM Models

Stages of LLM Training



Data Collection

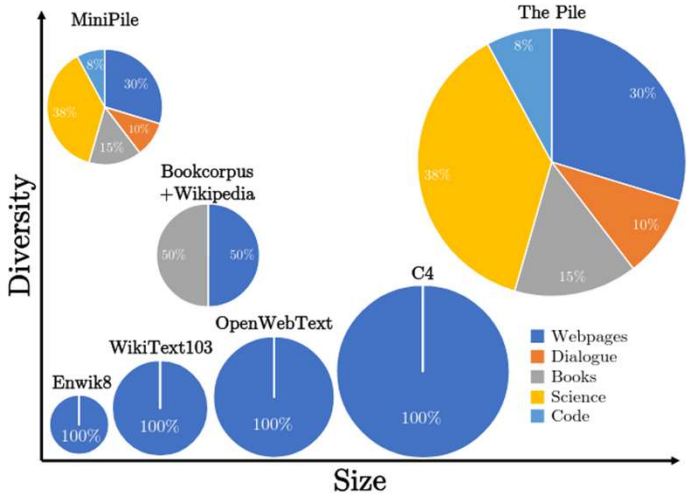
By crawling the open web, we can create massive datasets of all of the digital knowledge stored online.

The issues arise when finding what site to use and what not to use.

The **Common Crawl** project was started over a decade ago to find all addressable links.

Wikipedia also contains a huge repository of human language information

Getting the best data, though, is still an art rather than a science as the terabytes of raw text are impossible to manually clean and curate.



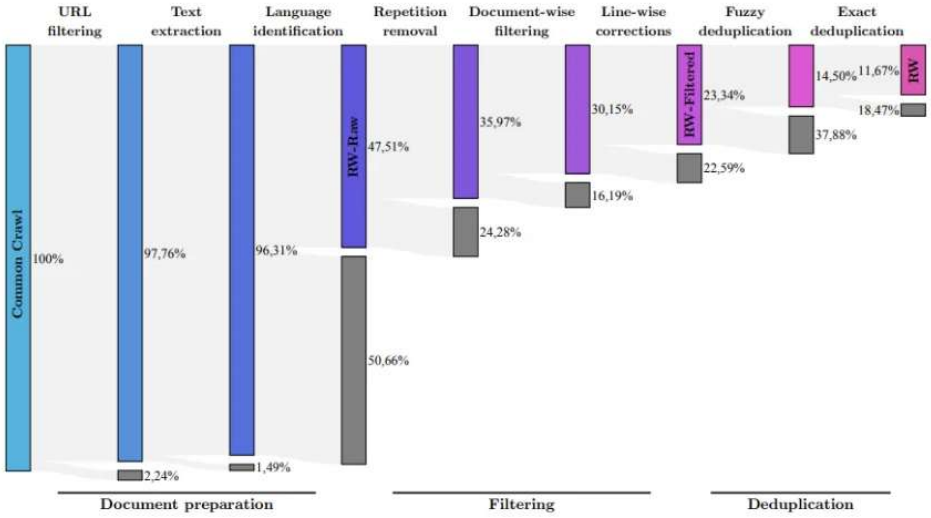
Dataset	Quantity (tokens)	Weight in training mix	Epochs elapsed when training for 300B tokens
Common Crawl (filtered)	410 billion	60%	0.44
WebText2	19 billion	22%	2.9
Books1	12 billion	8%	1.9
Books2	55 billion	8%	0.43
Wikipedia	3 billion	3%	3.4

Table 2.2: Datasets used to train GPT-3. "Weight in training mix" refers to the fraction of examples during training that are drawn from a given dataset, which we intentionally do not make proportional to the size of the dataset. As a result, when we train for 300 billion tokens, some datasets are seen up to 3.4 times during training while other datasets are seen less than once.

Source: Nvidia

Data Preprocessing

1. **Tokenization**: Breaking text into tokens (subworlds, words, or characters) for sequential modeling.
2. **Normalization**: Standardizing text (e.g., lowercase, removing special characters).
3. **Dataset Splitting**: Creating training, validation, and test sets for robust evaluation.
4. **Sequence Preparation**
   Generate input-output pairs:
   - Input: Sequence
   - Target: next token prediction
   - Padding/truncation for consistent sequence lengths in batches.

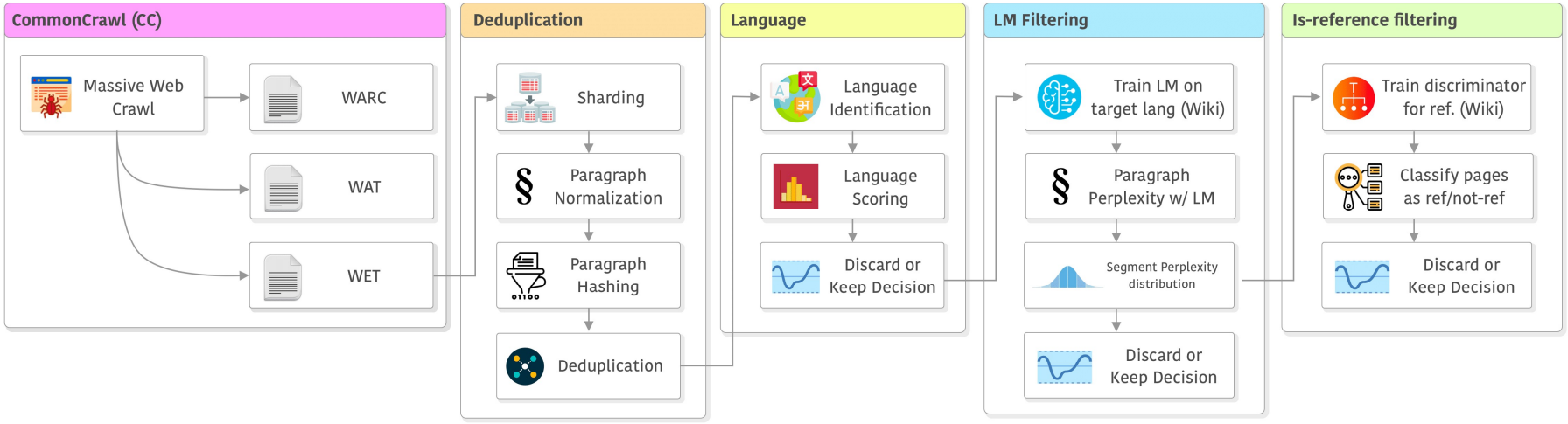


Source: Nvidia

Data Preprocessing

Once collected, the data must be processed before being useful for training. A typical workflow consists of:

1. Gather the raw data
2. Removing duplicated data (deduplication)
3. Filtering for language (e.g. only English)
4. Filtering for harmful or specific content
5. Remove unknown tokens
6. Format data into training and testing datasets

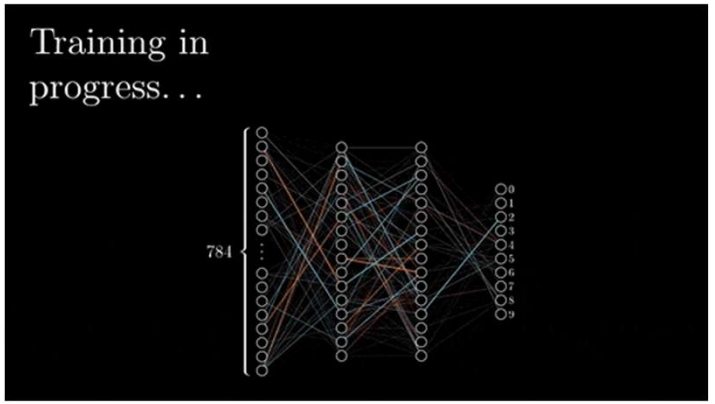


Source: Nvidia

Backpropagation for AR models

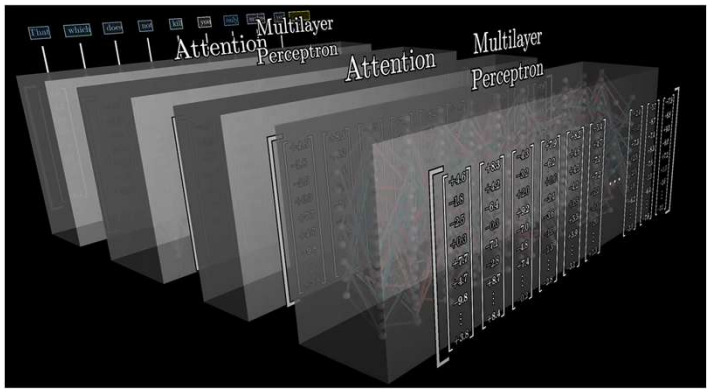
Forward Pass

1. **Input**: Token sequence
2. **Model Output**: Predicted probabilities for the next token
3. **Generate Loss**: How wrong a model's predicted probability is for the true answer, heavily penalizing wrong predictions.



Backward Pass: Gradient Computation

- Gradients are calculated layer-by-layer
  Starting from the loss at the output layer and propagating back through the model.
- Attention layers contribute gradients for both:
  - Key, Query, Value matrices
  - Multi-Headed Attention weights



Source: https://miro.medium.com/v2/resize:fit:828/format:webp/0*d9yJ5xIqdbDyjCYR.gif & https://www.youtube.com/watch?v=9-Jl0dxWQs8&list=PLZHQObOWTQDNU6R1_67000Dx_ZCJB-3pi&index=8

Loss Functions for AR Modeling

Loss Function

Measures how well the model predicts the next token in a sequence. The choice of loss function directly impacts the model's ability to generalize, handle uncertainty, and produce coherent outputs.

Cross-Entropy Loss

Measures the difference between predicted and true token distributions, giving a higher penalty when the model assigns low probability to the true outcome.

Label Smoothing

While Cross-Entropy Loss is the standard choice, cross-entry loss can lead to overconfident predictions. Enhancements like Label Smoothing are often used by distributing some probability mass to incorrect tokens.

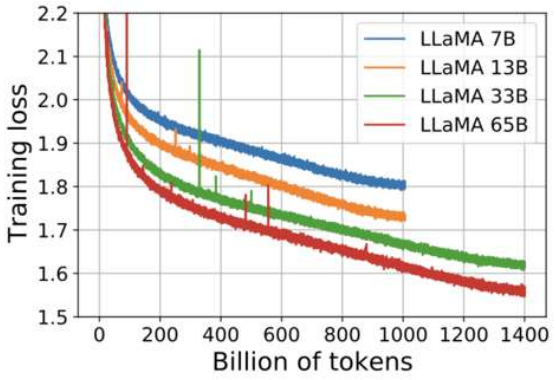


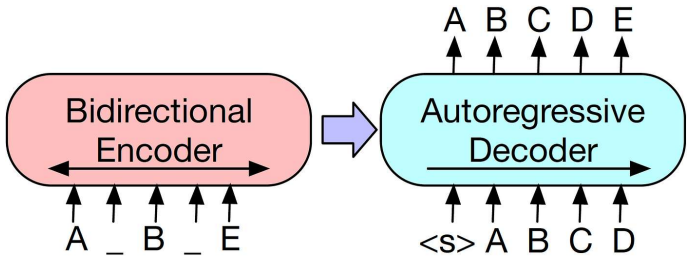
Figure 1: **Training loss over train tokens for the 7B, 13B, 33B, and 65 models.** LLaMA-33B and LLaMA-65B were trained on 1.4T tokens. The smaller models were trained on 1.0T tokens. All models are trained with a batch size of 4M tokens.

Source: https://miro.medium.com/v2/resize:fit:828/format:webp/0*d9yJ5xIqdbDyjCYR.gif & https://www.youtube.com/watch?v=9-Jl0dxWQs8&list=PLZHQObOWTQDNU6R1_67000Dx_ZCJB-3pi&index=8

Autoregressive Modeling in Language

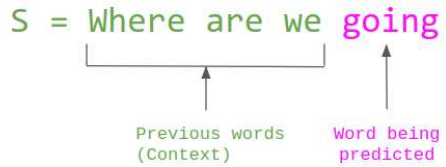
Encoder-Decoder Models

In the previous lectures we have seen that models can be used for language translation – using encoder-decoder models. These models processes tokens in parallel (e.g., BERT predicts missing words all at once).



Autoregressive Models

The Transformer model can be converted into what is known as an **autoregressive model** which is used to predict the next token (e.g. ChatGPT). Autoregressive Models predict the next token in a sequence given all previous tokens



Source: Nvidia

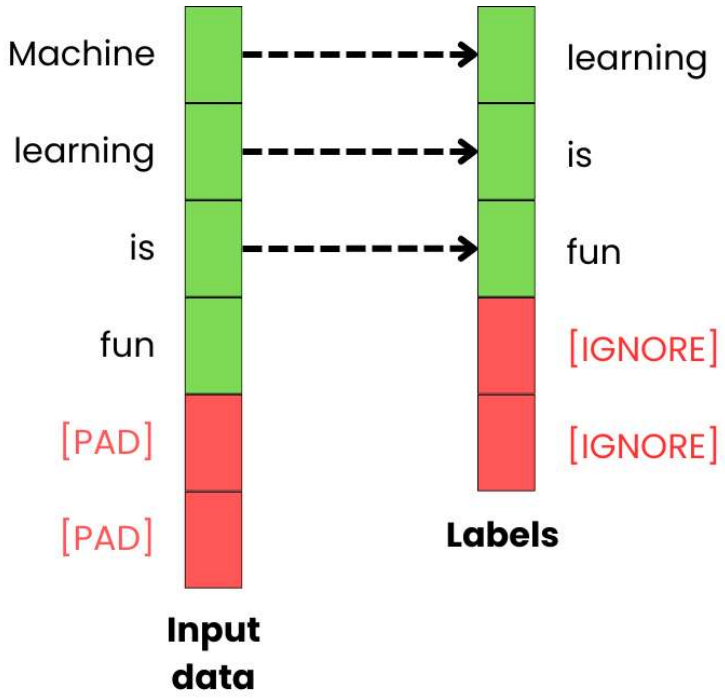
Autoregressive Models

In an autoregressive model, each value in a sequence is predicted based on the prior values.

Each value is predicted conditionally based on the previous values.

Autoregressive models are particularly powerful for sequential data and are fundamental to large language models (LLMs) like GPT (Generative Pre-trained Transformer), which predict tokens in a sequence of text.

The Autoregressive nature of LLMs explains a number of key advantages and challenges



Source: Nvidia

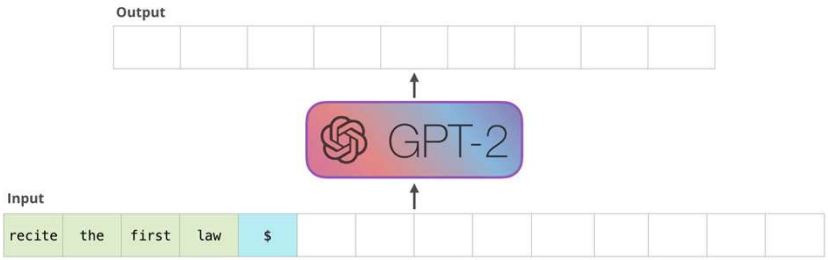
Limitations of base models

Shallow Understanding
Base models, like GPT, predict the next word based on patterns, not true comprehension. They generate plausible text without deep understanding.

No Task-Specific Knowledge
They struggle with complex tasks like reasoning, question-answering, or structured outputs because they aren't fine-tuned for specific tasks.

No Long-Term Planning:
Next-token models can't plan or keep track of long sequences. They may lose coherence in long tasks (like storytelling) because they don't retain a memory of earlier steps.

Limited in Complex Dependencies:
They handle local coherence but struggle with tasks requiring structure or logic over longer spans (e.g., math, formal reasoning).



Base Model
(davinci (GPT-3))

Prompt: Tell me how to fine-tune a model

Completion:
How can I control the complexity of a model?
How do I know when my model is done?
How do I test a model?
How do I define a task for my model?
How do I scale a model?
How do I make a probabilistic model?
How do I start using a model once it's complete?
How do I cope with missing data?
How do I make my modeling efforts sustainable?

Source: Nvidia

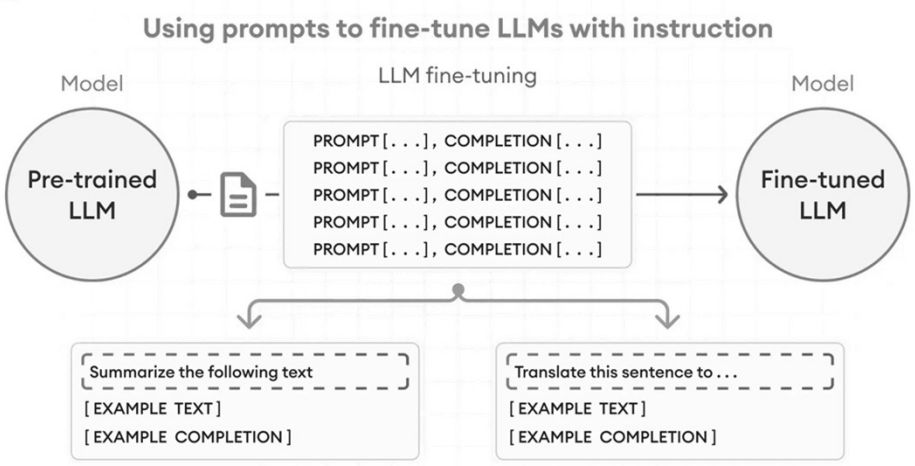

Instruction Tuning

Data Preparation

Generating prompt-response pairs requires more intentional curation than for pre-training.

The data can be generated synthetically by another LLM that has already been trained to create queries from raw data. More commonly or when starting from scratch, a human-generated dataset will need to be created with a variety of query-response pairs.

These pairs can be simple input-output or they can also include additional context to the query, e.g. Summarize this email into 5 bullet points.



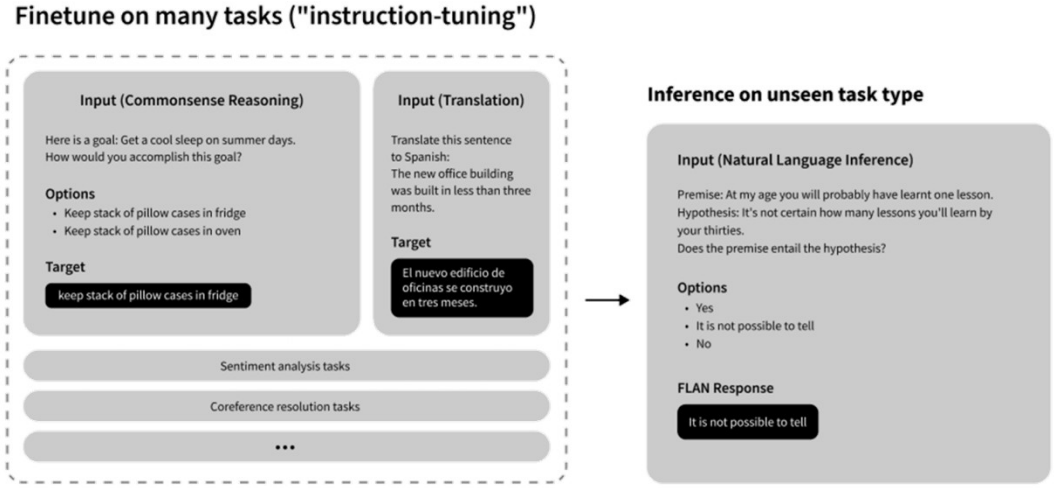
The model will be trained to produce the specific output, not predict the next token. This changes how the model behaves

Source: Nvidia

Instruction Tuning

Three parts:

1. **An instruction**
   A natural language text input that specifies a given task. For example, “translate this sentence from English to Spanish.”
2. **Additional information**
   Information that provides context relevant to the task at hand. For example, an input for a reading comprehension task might include a brief passage (and then instruct the model to answer a given question about it).
3. **Desired output**
   The target output—response—for the given prompt, per the instructions and context provided. This will serve as a ground truth against which the model’s predictions are evaluated and optimized.



Source: How to Train LLMs from Scratch from Weights & Biases



Reinforcement Learning

Reinforcement Learning vs Other Learning

Reinforcement Learning is a different class of learning. Rather than fitting an input-output relationship like supervised learning, RL models learn what actions should be take for a given scenario, this is called a **policy**

Supervised learning

Given labeled data: $\{(x_i, y_i)\}$, learn $f(x) \approx y$

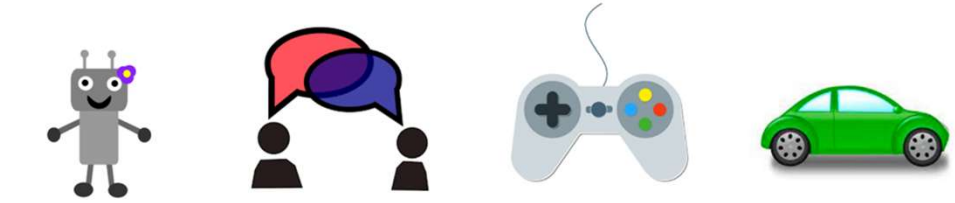
- directly told what to output
- inputs x are independently, identically distributed (i.i.d.)

Reinforcement learning

Learn *behavior* $\pi(a \mid s)$.

- from experience, indirect feedback
- data **not** i.i.d.: actions a affect the future observations.

Behavior can include:



motor control | dialog | game playing | driving

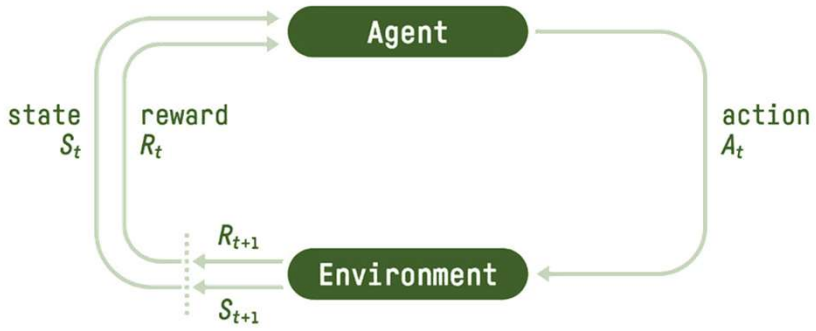
Source: Nvidia

Components of Reinforcement Learning

There are several components comprise an RL system.

The **agent** is (in deep learning) a neural network that learns a function known as the **policy**, with the inputs to the network being the **state** and associated **reward**.

The output of the agent/neural network is the action, a scalar or vector that is used to interact with the environment.
This is a digital representation (simulation, game, model) of the problem that the agent is trying to learn.



Term	Definition
Agent	An entity in a reinforcement learning scenario that makes decisions, typically a software entity or algorithm. The agent learns to choose actions that maximize some measure of long-term reward.
Environment	The external system or context with which the agent interacts. The environment presents states to the agent and responds to the agent's actions with changes to the state and/or rewards.
State (S)	A representation of the current situation or status within the environment. It's the information available to the agent at a particular time point for decision-making.
Action (A)	A set of possible moves or decisions that the agent can choose from in response to the current state. Actions can be discrete (specific choices) or continuous (a range of values).
Reward (R)	A scalar feedback signal given to the agent by the environment as a consequence of its actions. Rewards are used to assess the performance of the agent and guide its learning process.
Policy (π)	A strategy or rule followed by the agent to decide which action to take in a given state. It essentially maps states to actions and can be deterministic or stochastic.
Episode	A sequence of states, actions, and rewards that ends when reaching a terminal state. It's one complete playthrough or trial of the task at hand, from start to finish.

Source: Nvidia


Reinforcement Learning with Human Feedback

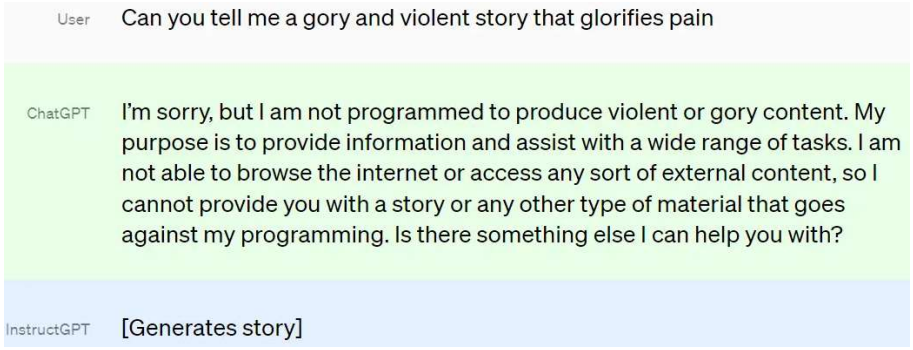
Why we need RLHF

A technique that uses human feedback to optimize ML models to self-learn more efficiently.

Reinforcement learning trains software to make decisions that maximize rewards, making their outcomes more accurate.

Alignment
RLHF incorporates human feedback in the rewards function, so the ML model can perform tasks more aligned with human goals, wants, and needs.

RLHF is used throughout generative artificial intelligence applications, including in large language models.



With vs. Without RLHF

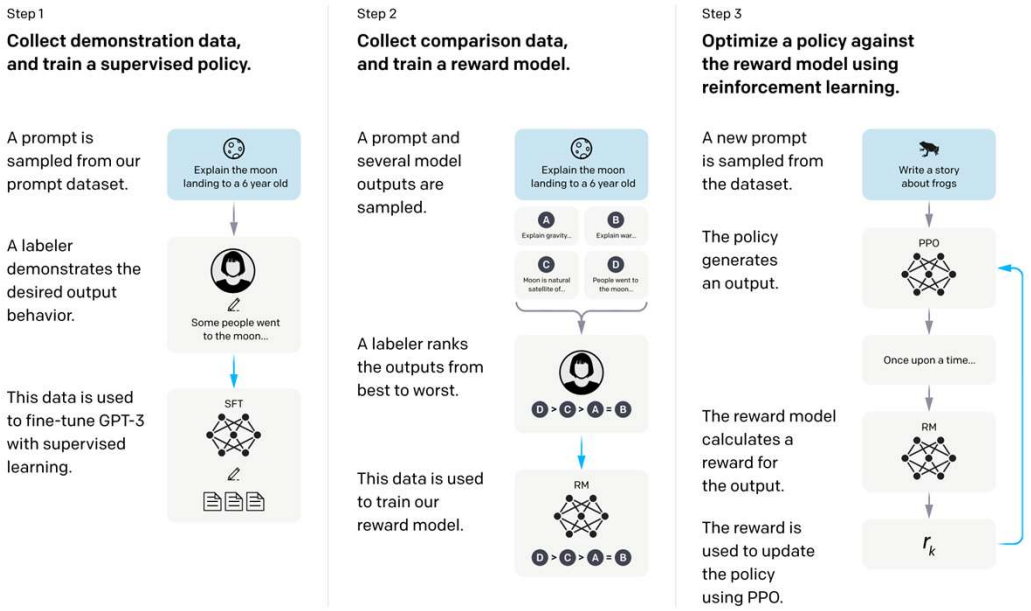
Source: Nvidia

Reinforcement Learning with Human Feedback (RLHF)

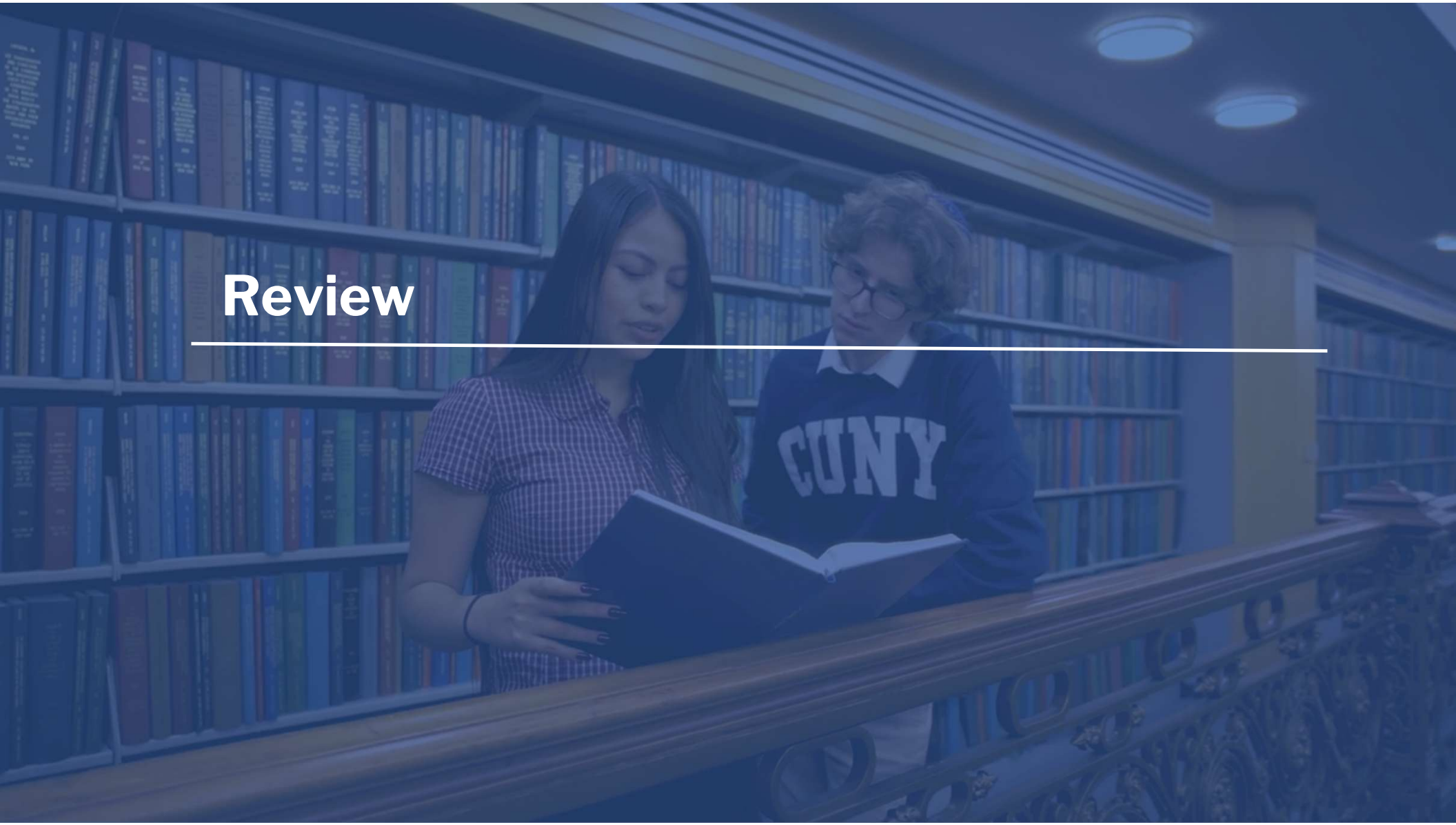
Forward Pass

Three steps:

1. Supervised fine-tuning (SFT)
2. Reward model (RM) training
3. Reinforcement learning via proximal policy optimization (PPO) on this reward model



Source: How to Train LLMs from Scratch from Weights & Biases



Review

This week we covered

Lesson Objectives/Topics

- **Foundational**
  - Data sources and pretraining
  - Supervised fine-tuning (SFT)
  - Reinforcement Learning
  - Reinforcement Learning with Human Feedback (RLHF)


CUNY | School of
Professional Studies